

Computadores multicore

18.1 Questões sobre desempenho de hardware

- Aumento em paralelismo
- Consumo de energia

18.2 Questões sobre desempenho de software

- Software em multicore
- Exemplo de aplicação: software de jogo da Valve

18.3 Organização multicore

18.4 Organização multicore x86 da Intel

- Intel Core Duo
- Intel Core i7

18.5 ARM11 MPCore

- Tratamento de interrupções
- Coerência de cache

18.6 Leitura recomendada e sites Web

- Sites web recomendados

PRINCIPAIS PONTOS

- Um computador multicore, ou chip multiprocessador, combina dois ou mais processadores em um único chip de computador.
- O uso de chips com processador único cada vez mais complexo atingiu o limite por conta do desempenho do hardware, incluindo limites no paralelismo em nível de instruções e limitações de energia.
- Por outro lado, a arquitetura multicore oferece desafios para desenvolvedores de software para explorar a capacidade do multithreading por meio de vários núcleos.
- As principais variáveis em uma organização multicore são o número de processadores no chip, o número de níveis da memória cache e a extensão em que a memória cache é compartilhada.
- Outra decisão sobre projeto organizacional em um sistema multicore é se os núcleos individuais serão superescalares ou se implementarão multithreading simultâneo (SMT).

Um computador **multicore**, conhecido também como **chip multiprocessador**, combina dois ou mais processadores (chamados núcleos — *core*) em uma única peça de silício (chamada pastilhas — *die*). Normalmente, cada núcleo consiste de todos os componentes de um processador independente, como registradores, ALU, hardware de pipeline e unidade de controle, mais caches L1 de dados e de instruções. Além de vários núcleos, os chips multicore atuais incluem também cache L2 e, em alguns casos, cache L3.

Este capítulo fornece uma visão geral sobre sistemas multicore. Começamos com uma análise sobre fatores de desempenho de hardware que levaram ao desenvolvimento de computadores multicore e dos desafios de software de explorar o poder de um sistema multicore. A seguir, analisamos a organização multicore. Finalmente, examinamos dois exemplos de produtos multicore, um da Intel e outro da ARM.

18.1 Questões sobre desempenho de hardware

Conforme discutimos no Capítulo 2, os sistemas dos microprocessadores experimentaram um aumento sólido e exponencial do desempenho de execução durante décadas. A Figura 2.12 mostra que este aumento

deve-se parcialmente aos refinamentos na organização do processador no chip e parcialmente ao aumento na frequência de clock.



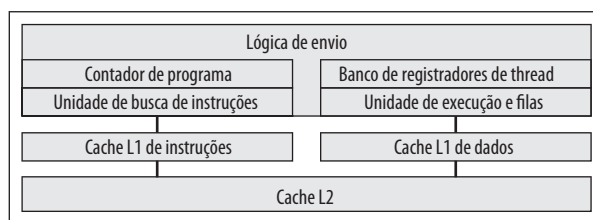
Aumento em paralelismo

As mudanças organizacionais no projeto dos processadores se concentraram, em primeiro lugar, no aumento do paralelismo em nível de instruções, para que mais trabalho pudesse ser feito em cada ciclo de clock. Estas mudanças incluem, em ordem cronológica (Figura 18.1):

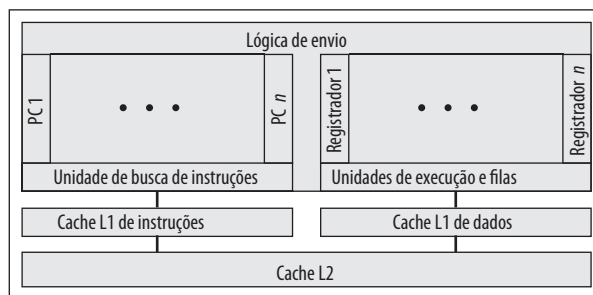
- **Pipeline:** instruções individuais são executadas por um pipeline de estágios de tal forma que, durante a execução de uma instrução em um estágio do pipeline, outra instrução é executada em outro estágio do pipeline.
- **Superscalar:** vários pipelines são construídos pela replicação de recursos da execução. Isto possibilita execução paralela de instruções em pipelines paralelos, assim que os hazards são evitados.
- **Multithreading simultâneo (SMT):** bancos de registradores são replicados para que várias threads possam compartilhar o uso dos recursos do pipeline.

Para cada uma destas inovações, os projetistas tentaram, ao longo dos anos, aumentar o desempenho do sistema acrescentando a complexidade. No caso do uso de pipeline, pipelines simples de três estágios foram substituídos

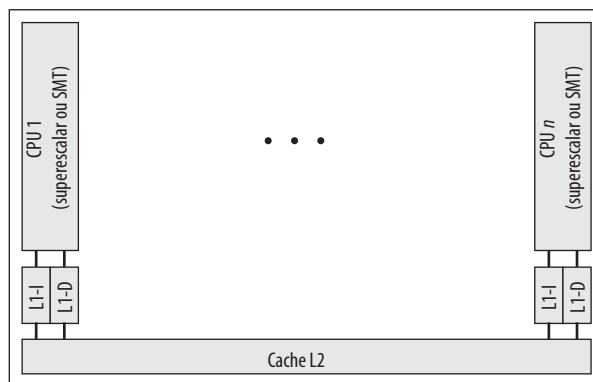
Figura 18.1 Organizações alternativas do chip



(a) Superscalar



(b) Multithreading simultâneo

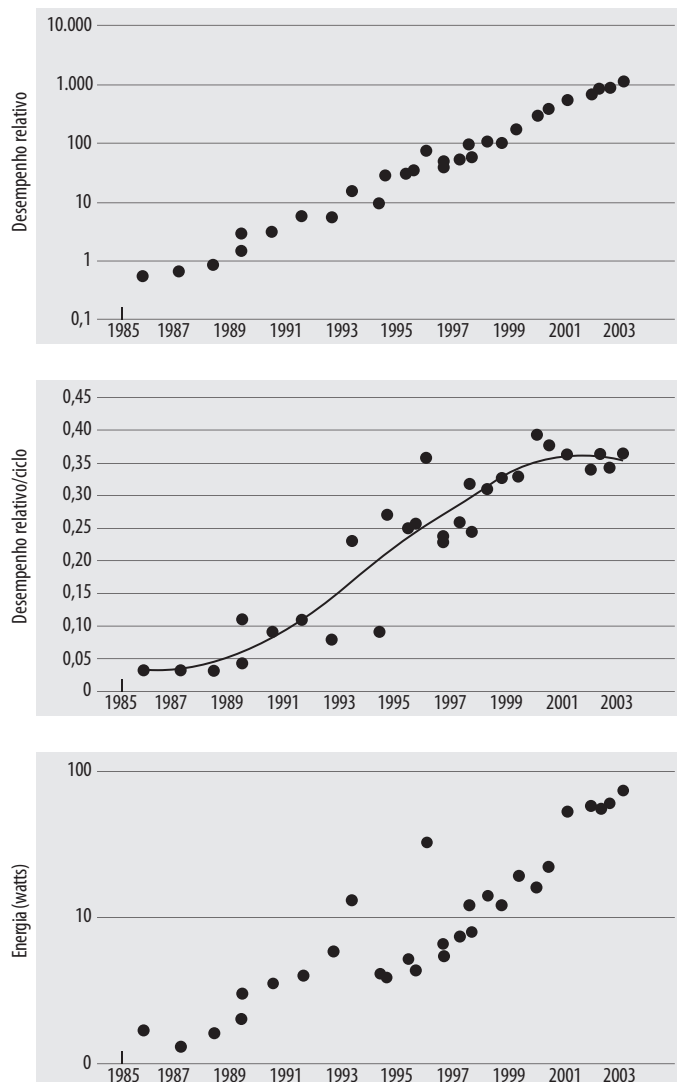


(c) Multicore

pelos pipelines com cinco estágios e depois com muito mais estágios, com algumas implementações tendo mais de doze estágios. Há um limite prático para até onde essa tendência pode ser levada porque, com mais estágios, há necessidade por mais lógica, mais interconexões e mais sinais de controle. Com a organização superescalar, aumentos de desempenho podem ser alcançados ao se aumentar o número de pipelines paralelos. Novamente, o retorno diminui à medida que o número de pipelines aumenta. Mais lógica é necessária para gerenciar os hazards e para recursos do estágio de instruções. Eventualmente, uma única thread de execução alcança o ponto onde os hazards e dependências de recursos impedem o uso total de vários pipelines disponíveis. Este mesmo ponto da diminuição de retornos acontece com o SMT, à medida que a complexidade de gerenciar várias threads por meio de um conjunto de pipelines limita o número de threads e o número de pipelines que podem ser usados efetivamente.

A Figura 18.2, retirada de Olukotun e Hammond (2005^a), é ilustrativa neste contexto. O gráfico superior mostra o aumento exponencial do desempenho dos processadores Intel ao longo dos anos.¹ O gráfico do meio é calculado ao se combinarem figuras publicadas de SPEC CPU da Intel e a frequência de clock dos processadores para se ter uma medida de quanto da melhoria de desempenho se deve ao aumento de exploração do paralelismo em nível

Figura 18.2 Algumas tendências do hardware da Intel



¹ Os dados são baseados em figuras SPEC CPU publicadas pela Intel e normalizadas em vários conjuntos.

de instruções. Há uma região achatada no começo dos anos 1980 antes de o paralelismo ser explorado extensivamente. Segue-se a isso uma forte subida quando os projetistas foram capazes de explorar substancialmente o pipeline, técnicas superescalares e o SMT. Mas, no início de 2000, uma nova região achatada na curva aparece, à medida que foram sendo alcançados os limites da exploração de paralelismo em nível de instruções.

Há um conjunto de problemas relatado que trata de questões relacionadas ao projeto e à fabricação de chips de computadores. O aumento na complexidade para lidar com todas as questões de lógica relacionadas com pipelines muitos longos, vários pipelines superescalares e vários bancos de registradores SMT significa que uma grande área do chip é ocupada com lógica de coordenação e transferência de sinais. Isso aumenta a dificuldade de projeto, fabricação e depuração de chips. O difícil e crescente desafio de engenharia relacionado à lógica do processador é uma das razões pelo aumento de uma parte do chip dedicada à lógica de memória mais simples. Questões de energia, discutidas a seguir, dão outra razão.



Consumo de energia

Para manter a tendência de desempenho mais alta à medida que o número de transistores por chip aumenta, os projetistas recorreram aos projetos de processadores mais elaborados (pipeline, superescalar, SMT) e às altas frequências de clock. Infelizmente, requisitos de energia cresceram exponencialmente à medida que aumentaram a densidade e a frequência de clock do chip. Isso é mostrado no gráfico inferior da Figura 18.2

Uma maneira de controlar densidade da energia é usar mais área do chip para memória cache. Os transistores são menores e têm uma densidade de energia em ordem de magnitude menor do que a da lógica (veja Figura 18.3a). Conforme mostra a Figura 18.3b (BORKAR, 2003^b), a porcentagem da área do chip dedicada à memória cresceu para mais de 50% à medida que a densidade de transistores do chip aumentou.

A Figura 18.4, retirada de Borkar (2007^c), mostra para onde está indo a tendência do consumo de energia. Até 2015, podemos esperar ver chips de microprocessadores com cerca de 100 bilhões de transistores em um molde de 300 mm². Supondo que 50 a 60% da área do chip seja dedicado à memória, ele suportará memória cache em torno de 100 MB e deixará em torno de 1 bilhão de transistores disponíveis para lógica.

Como usar todos esses transistores é um ponto-chave do projeto. Conforme discutido anteriormente nesta seção, existe limite para uso efetivo de tais técnicas como superescalar e SMT. Em termos gerais, a experiência das décadas recentes foi definida por uma regra conhecida como **regra de Pollack** (POLLACK, 1999^d), que diz que o aumento de desempenho é diretamente proporcional à raiz quadrada do aumento de complexidade. Em outras palavras, se você dobrar a lógica em um núcleo do processador, então ele apresenta apenas 40% a mais de desempenho. A princípio, o uso de vários núcleos tem o potencial para fornecer um aumento de desempenho quase linear com aumento em número de núcleos.

Figura 18.3 Considerações sobre energia e memória

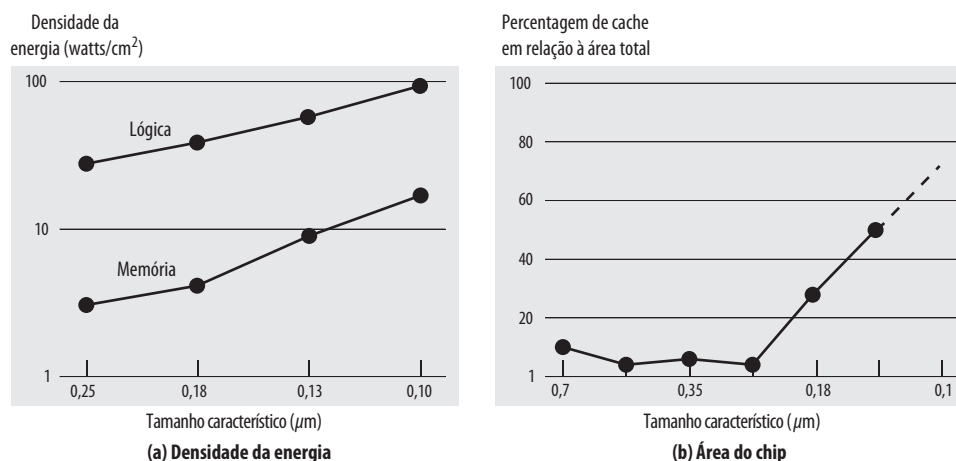
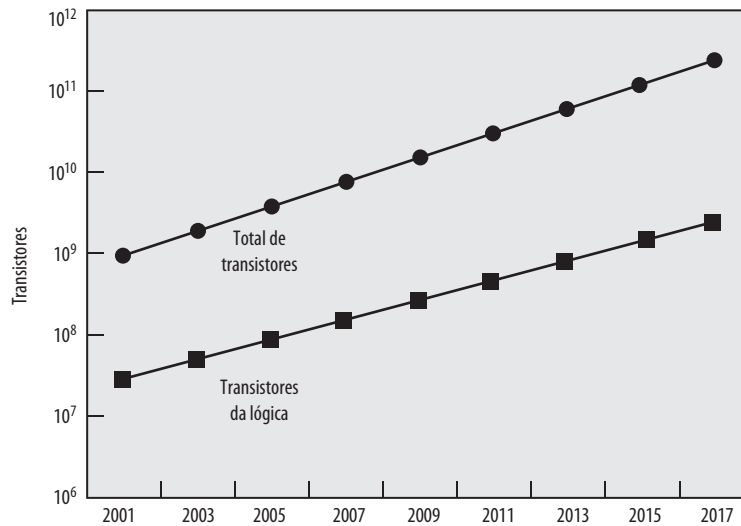


Figura 18.4 Utilização de transistores do chip

Considerações sobre energia fornecem outro motivo para ir em direção a uma organização multicore. Como o chip tem tanta quantidade de memória cache, torna-se improvável que uma única thread de execução possa efetivamente usar toda essa memória. Mesmo com SMT, você está fazendo multithreading de forma relativamente limitada e não pode, portanto, explorar totalmente uma cache gigante, enquanto que um número de threads ou processos relativamente independentes tem uma oportunidade maior de obter a total vantagem da memória cache.



18.2 Questões sobre desempenho de software

Uma análise mais detalhada sobre questões de desempenho de software relacionada à organização multicore está além do nosso escopo. Nesta seção, primeiro fornecemos uma visão geral dessas questões e depois analisamos um exemplo de uma aplicação projetada para explorar capacidades multicore.



Software em multicore

Os benefícios potenciais de desempenho de uma organização multicore dependem da habilidade de explorar efetivamente os recursos paralelos disponíveis para a aplicação. Vamos focar primeiro em uma única aplicação executando em um sistema multicore. Relembre do Capítulo 2 que a lei de Amdahl afirma que:

$$\begin{aligned} \text{Aumento de velocidade} &= \frac{\text{tempo para executar o programa em um único processador}}{\text{tempo para executar o programa em } N \text{ processadores paralelos}} \\ &= \frac{1}{(1 - f) + \frac{f}{N}} \end{aligned} \quad (18.1)$$

A lei supõe um programa no qual uma fração $(1 - f)$ do tempo de execução envolve o código que é inerentemente serial e uma fração f que envolve código infinitamente paralelizável com nenhuma sobrecarga de escalonamento.

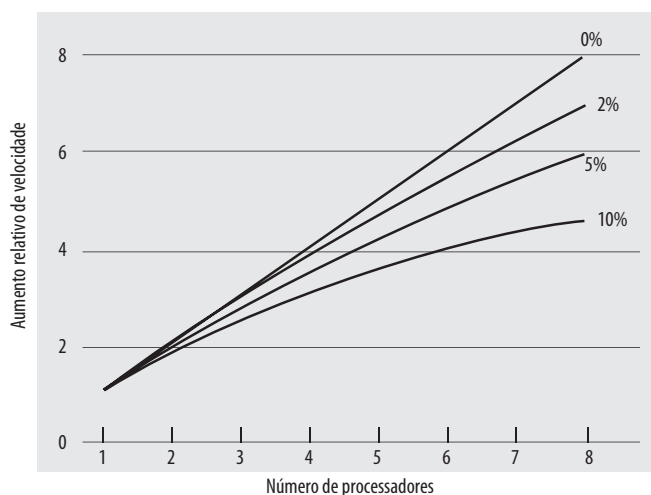
Essa lei surge para tornar mais atraente a possibilidade de uma organização multicore. Mas, como mostra Figura 18.5a, até uma quantidade pequena de código serial tem um impacto notável. Se apenas 10% do código for inerentemente serial ($f = 0,9$), executar o programa em um sistema multicore com 8 processadores produz um ganho de desempenho de um fator de apenas 4,7. Além disso, o software normalmente provoca sobrecarga

como o resultado de comunicação e a distribuição de trabalho para vários processadores e sobrecarga de coerência de cache. Isso resulta em uma curva na qual o desempenho alcança picos e depois começa a degradar por causa do aumento da sobrecarga de uso de vários processadores. A Figura 18.5b (MCDUGALL, 2005^e) é um exemplo representativo.

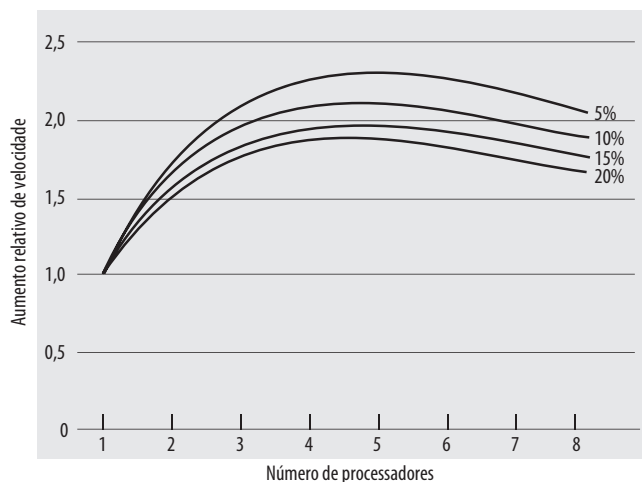
No entanto, engenheiros de software resolveram esse problema e existem várias aplicações em que é possível explorar efetivamente um sistema multicore. McDougall (2005^e) reporta um conjunto de aplicações de banco de dados onde grande atenção foi dedicada a reduzir a fração serial dentro de arquiteturas de hardware, sistemas operacionais, middleware e softwares de banco de dados. A Figura 18.6 mostra o resultado. Conforme mostra este exemplo, os sistemas de gerenciamento de banco de dados e aplicações de banco de dados são uma área em que os sistemas multicore podem ser usados efetivamente. Vários tipos de servidores também podem usar efetivamente a organização multicore paralela, porque os servidores normalmente lidam com numerosas transações relativamente independentes em paralelo.

Além do software de propósito geral para servidores, uma série de tipos de aplicações se beneficia diretamente da habilidade de dimensionar rendimento de acordo com o número de núcleos. McDougall e Laudon (2006^f) listam os seguintes exemplos:

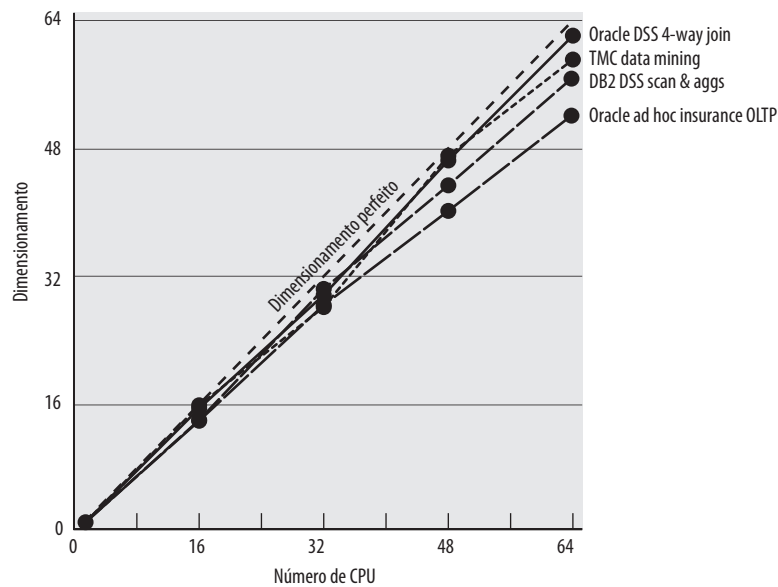
Figura 18.5 Efeitos de desempenho de múltiplos cores



(a) Aumento de velocidade com partes sequenciais de 0%, 2%, 5% e 10%



(b) Aumento de velocidade com sobrecargas

Figura 18.6 Dimensionamento de trabalhos de bancos de dados em hardware com vários processadores

- **Aplicações multithread nativas:** aplicações multithread são caracterizadas por ter um pequeno número de processos com ótimas condições para o uso de multithread. Exemplos de aplicações com ótimas condições para o uso de multithread incluem Lotus Domino ou Siebel CRM (Customer Relationship Manager).
- **Aplicações com múltiplos processos:** aplicações com múltiplos processos são caracterizadas pela presença de muitos processos de thread única. Exemplo de aplicações com múltiplos processos incluem banco de dados Oracle, SAP e PeopleSoft.
- **Aplicações Java:** aplicações Java abraçam threads de uma maneira fundamental. Não apenas a linguagem Java facilita muito aplicações multithread, mas a Java Virtual Machine é um processo multithread que provê agendamento e gerenciamento de memória para aplicações Java. Aplicações Java podem, então, se beneficiar diretamente dos recursos multicore incluindo servidores de aplicação como Java Application Server da Sun, Weblogic da BEA, Websphere da IBM e servidor de aplicação de código fonte aberto Tomcat. Todas as aplicações que usam um servidor de aplicações da plataforma Java 2 Enterprise Edition (plataforma J2EE) podem se beneficiar imediatamente da tecnologia multicore.
- **Aplicações com múltiplas instâncias:** mesmo que uma aplicação individual não possa ser dimensionada para obter vantagem de um número grande de threads, ainda é possível se beneficiar da arquitetura multicore executando várias instâncias da aplicação em paralelo. Se várias instâncias de aplicação requerem algum tipo de isolamento, a tecnologia de virtualização (para o hardware do sistema operacional) pode ser usada para fornecer a cada uma delas o seu próprio ambiente separado e seguro.



Exemplo de aplicação: software de jogo da Valve

A Valve é uma empresa de entretenimento e tecnologia que desenvolveu uma série de jogos populares, assim como o motor Source, um dos motores de jogos disponíveis mais usados. Source é um motor de animação usado pela Valve para seus jogos e licenciado para outros desenvolvedores de jogos.

Nos últimos anos, a Valve reprogramou o software do motor Source para usar multithreading a fim de explorar a capacidade dos chips de processadores multicore da Intel e AMD (REIMER, 2006⁹). O código revisado do motor Source fornece suporte mais poderoso para jogos da Valve como Half Life 2.

Da perspectiva da Valve, as opções de granularidade de threads são definidas a seguir (HARRIS, 2006^h):

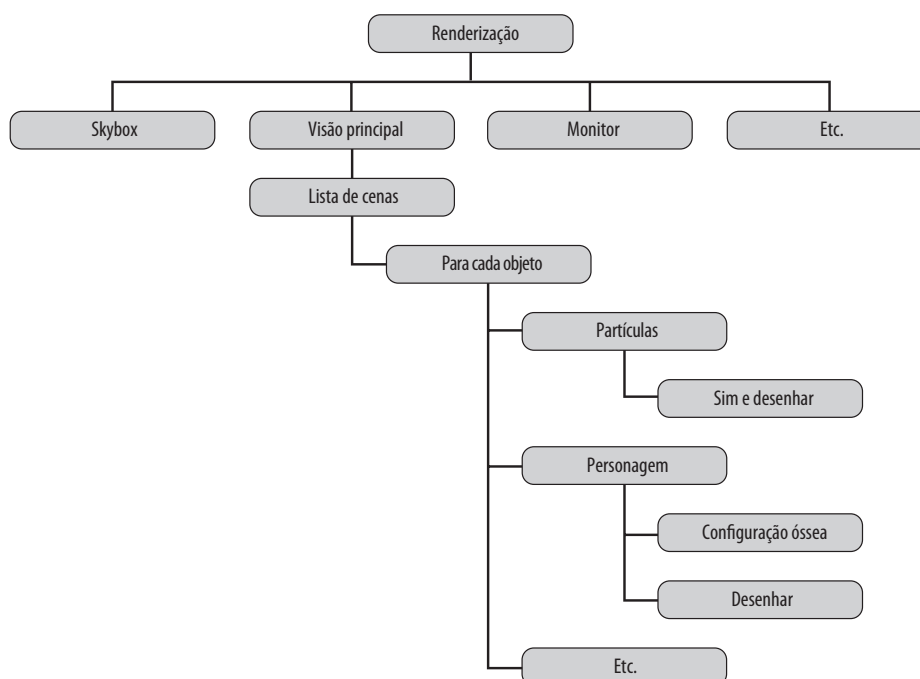
- **Granularidade grossa de threads:** módulos individuais, chamados de sistemas, são atribuídos a processadores individuais. No caso do motor Source, isso significaria colocar renderização em um processador, IA (inteligência artificial) no outro, física no outro, e assim por diante. Isto é bem direto. Basicamente, cada módulo maior é uma única thread e a coordenação principal envolve sincronizar todas as threads com uma thread da linha de tempo.
- **Granularidade fina de threads:** muitas tarefas semelhantes ou idênticas são espalhadas por vários processadores. Por exemplo, um laço que faz iteração sobre uma matriz de dados pode ser dividido em um número de laços menores em threads individuais que podem ser agendadas em paralelo.
- **Thread híbrido:** isto envolve o uso seletivo de threads de granularidade fina para alguns sistemas e threads únicas para outros sistemas.

A Valve concluiu que, por meio da granularidade grossa, poderia alcançar até o dobro do desempenho em dois processadores quando comparado com execução em um único processador. Mas este ganho de desempenho apenas poderia ser alcançado com casos artificiais. Para jogos do mundo real, a melhoria estava na ordem de um fator de 1,2. A empresa também concluiu que o uso efetivo de granularidade fina era difícil. O tempo por unidade de trabalho pode ser variável e gerenciar a linha de tempo de saídas e as consequências envolvia programação complexa.

A Valve concluiu que uma abordagem de thread híbrida era a mais promissora e seria mais bem dimensionada à medida que sistemas multicore com 8 ou 16 processadores se tornassem disponíveis. Ela identificou sistemas que operam com muita eficiência, sendo permanentemente atribuídos a um único processador. Um exemplo é a mixagem de som, a qual tem pouca interação do usuário, não é restringida pela configuração do quadro das janelas e funciona no seu próprio conjunto de dados. Outros módulos, como renderização de cenas, podem ser organizados em um número de threads para que o módulo possa executar em um único processador, mas que possa alcançar desempenho melhor quando é espalhado por mais e mais processadores.

A Figura 18.7 ilustra a estrutura de threads para o módulo de renderização. Nesta estrutura hierárquica, threads de níveis mais altos geram threads de níveis mais baixos conforme necessário. O módulo de renderização depende de uma parte crítica do motor Source, a lista mundial, que é um banco de dados que representa os elementos visuais no mundo dos jogos. A primeira tarefa é determinar quais são as áreas do mundo que precisam ser renderi-

Figura 18.7 Threading híbrido para módulo de renderização



zadas. A próxima tarefa é determinar quais objetos estão em cena conforme vistos de vários ângulos. Depois vem o trabalho intensivo do processador. O módulo de renderização tem que trabalhar a renderização de cada objeto de vários pontos de vista, como visão do jogador, visão dos monitores de TV e o ponto de vista dos reflexos na água.

Alguns dos principais elementos da estratégia de thread para o módulo de renderização estão relacionados em Leonard (2007) e incluem o seguinte:

- Construir listas de renderização de cenas para várias cenas em paralelo (por exemplo, o mundo e o seu reflexo na água).
- Sobrepor simulação dos gráficos.
- Transformação do personagem computacional para todos os personagens em todas as cenas em paralelo.
- Permitir que várias threads desenhem em paralelo.

Os projetistas descobriram que simplesmente bloquear os principais bancos de dados, como a lista mundial, para uma thread era ineficiente demais. Em mais de 95% do tempo, uma thread está tentando ler de um conjunto de dados e apenas 5% do tempo, no máximo, é gasto escrevendo no conjunto de dados. Assim, um mecanismo de concorrência conhecido como modelo escritor-único-múltiplos-leitores funciona eficientemente.



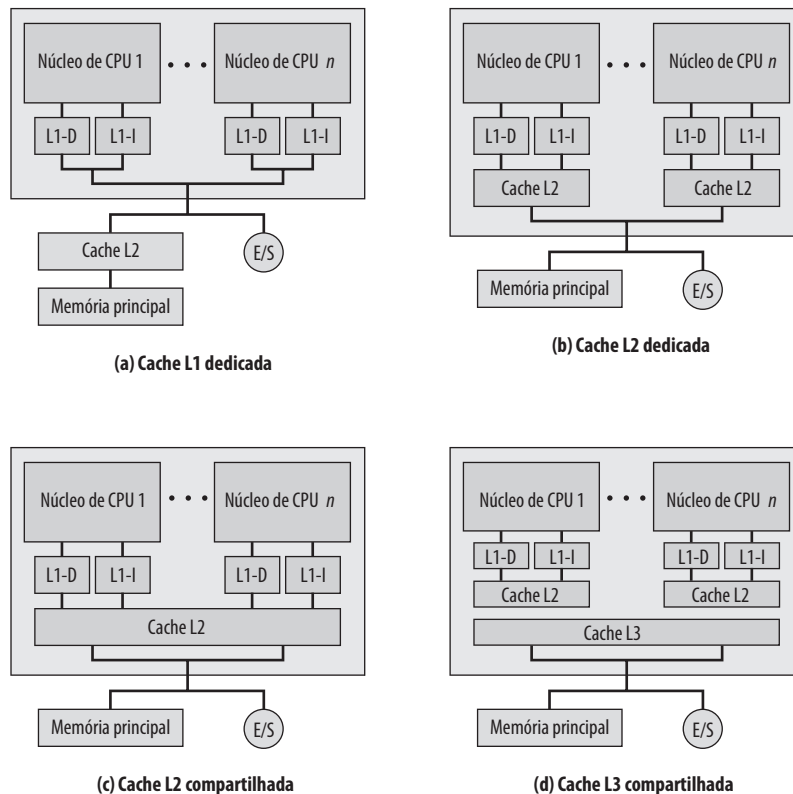
18.3 Organização multicore

No nível mais alto da descrição, as principais variáveis em uma organização multicore são as seguintes:

- Número de núcleos processadores no chip.
- Número de níveis da memória cache.
- Quantidade de memória cache que é compartilhada.

A Figura 18.8 mostra quatro organizações gerais para sistemas multicore. A Figura 18.8a é uma organização encontrada em alguns computadores com chips multicore anteriores e encontra-se ainda nos chips embutidos.

Figura 18.8 Alternativas da organização multicore



Nesta organização, a única cache no chip é L1, com cada núcleo tendo a sua cache L1 dedicada. Quase invariavelmente, a cache L1 é dividida em caches de dados e instruções. Um exemplo desta organização é ARM11 MPCore.

A organização da Figura 18.8b é também uma onde não há compartilhamento da cache no chip. Neste caso, há bastante área disponível no chip para permitir a cache L2. Um exemplo desta organização é o AMD Opteron. A Figura 18.8c mostra uma alocação semelhante de espaço do chip para memória, porém com uso de cache L2 compartilhada. O Core Duo da Intel tem essa organização. Finalmente, à medida que a quantidade de memória cache disponível no chip continua a crescer, as considerações sobre desempenho ditam a divisão de uma cache L3 separada e compartilhada, com caches L1 e L2 dedicadas para cada núcleo do processador. O Core i7 da Intel é um exemplo desta organização.

O uso de uma cache L2 compartilhada na cache tem várias vantagens em relação à dependência exclusiva das caches dedicadas:

1. Interferência construtiva pode reduzir as taxas gerais de falhas. Ou seja, se uma thread em um núcleo acessa uma posição da memória principal, isso traz um quadro contendo a posição referenciada para a cache compartilhada. Se uma thread em outro núcleo acessar logo depois o mesmo bloco de memória, as posições de memória já estarão disponíveis na cache compartilhada no chip.
2. Uma vantagem relacionada é que dados compartilhados por vários núcleos não são replicados em nível de cache compartilhada.
3. Com algoritmos adequados de substituição de quadros, a quantidade de cache compartilhada alocada para cada núcleo é dinâmica, para que as threads que têm menos espaço possam empregar mais cache.
4. A comunicação entre processadores é fácil de implementar por meio das posições de memória compartilhadas.
5. O uso de uma cache L2 compartilhada confina o problema de coerência de cache para o nível da cache L1, o que pode acarretar algumas vantagens adicionais para o desempenho.

Uma vantagem potencial em se ter apenas caches L2 dedicadas no chip é que cada núcleo usufrui de acesso mais rápido à sua cache L2 privada. Isto é vantajoso para threads que têm forte localidade.

À medida que a quantidade de memória disponível e o número de núcleos crescem, o uso de uma cache L3 compartilhada combinada com uma cache L2 compartilhada ou cache L2 dedicada por núcleo tende a fornecer um desempenho melhor do que simplesmente uma cache L2 compartilhada massivamente.

Outra decisão de projeto organizacional em um sistema multicore é se os núcleos individuais serão superescalares ou se implementarão multithread simultâneo (SMT). Por exemplo, o Core Duo da Intel usa núcleos superescalares, enquanto o Core i7 da Intel usa núcleos SMT. O SMT tem o efeito de aumentar o número de threads em nível de hardware que o sistema multicore suporta. Assim, um sistema multicore com quatro núcleos e SMT que suporta quatro threads simultâneas em cada núcleo aparece para o nível da aplicação da mesma forma, como um sistema multicore com 16 núcleos. À medida que o software é desenvolvido para explorar mais profundamente recursos paralelos, uma abordagem SMT parece ser mais atraente do que uma abordagem superescalar.



18.4 Organização multicore x86 da Intel

A Intel introduziu uma série de produtos multicore nos últimos anos. Nesta seção analisamos dois exemplos: o Intel Core Duo e o Intel Core i7.

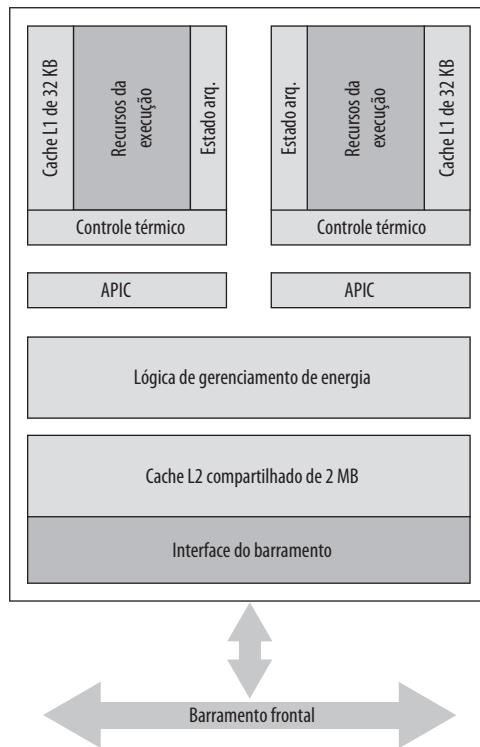


Intel Core Duo

O Intel Core Duo, introduzido em 2006, implementa dois processadores x86 superescalares com uma cache L2 compartilhada (Figura 18.8c).

A estrutura geral de Intel Core Duo é mostrada na Figura 18.9. Vamos considerar os principais elementos começando pelo topo da figura. Como é comum em sistemas multicore, cada núcleo tem a sua **cache L1** dedicada. Neste caso, cada núcleo tem uma cache de instruções de 32 KB e uma cache de dados de 32 KB.

Cada núcleo tem uma **unidade de controle térmica**. Com a densidade maior de chips atuais, o gerenciamento térmico é uma capacidade fundamental, especialmente para laptops e sistemas móveis. A unidade de controle térmico de Core Duo é projetada para gerenciar a dissipação de calor do chip para maximizar o desempenho do

Figura 18.9 Diagrama de blocos do Intel Core Duo

processador dentro das restrições térmicas. O gerenciamento térmico melhora também a ergonomia para o sistema de esfriamento e menor barulho acústico do ventilador. Basicamente, a unidade de gerenciamento térmico monitora sensores digitais de alta precisão para medição de temperatura de alta precisão. Cada núcleo pode ser definido como uma zona térmica independente. A temperatura máxima para cada zona térmica é reportada separadamente por meio registradores dedicados que podem ser consultados pelo software. Se a temperatura em um núcleo exceder o limite, a unidade de controle térmica reduz a taxa de clock para diminuir a geração de calor.

O próximo elemento-chave da organização Core Duo é o **controlador programável avançado de interrupções** (APIC, do inglês *advanced programmable interrupt controller*). O APIC desempenha uma série de funções, incluindo as seguintes:

1. O APIC pode prover interrupções entre processadores, o que permite que qualquer processador interrompa qualquer outro processador ou conjunto de processadores. No caso do Core Duo, uma thread em um núcleo pode gerar uma interrupção que é aceita pelo APIC local, encaminhada para o APIC do outro núcleo e comunicada como uma interrupção para outro núcleo.
2. O APIC aceita interrupções de E/S e encaminha-as para o núcleo adequado.
3. Cada APIC inclui um temporizador, o qual pode ser ajustado pelo SO para gerar uma interrupção no núcleo local.

A **lógica de gerenciamento de energia** é responsável por reduzir o consumo de energia quando possível, aumentando a vida das baterias em plataformas móveis, como laptops. Basicamente, a lógica de gerenciamento de energia monitora as condições térmicas e a atividade da CPU e ajusta os níveis de voltagem e consumo de energia de acordo. Ela inclui uma capacidade avançada de chaveamento de energia que possibilita um controle lógico de granularidade muito fina que liga subsistemas lógicos individuais do processador apenas se e quando eles forem necessários. Além disso, muitos barramentos e vetores são divididos para que os dados necessários em alguns modos de operação possam ser colocados em estado de energia baixo quando não são necessários.

O chip Core Duo inclui uma **cache L2** de 2 MB compartilhada. A lógica da cache permite alocação dinâmica do espaço da cache com base nas necessidades atuais do núcleo, de tal forma que a um núcleo possa ser atribuído até

100 por cento de cache L2. A cache L2 inclui a lógica para suportar o protocolo MESI para caches L1 anexadas. O principal ponto para ser considerado é quando uma escrita é feita em nível L1. Uma linha de cache obtém o estado M quando um processador escreve nela; se a linha não estiver no estado E ou M antes de escrever nela, a cache envia uma requisição Leitura-Para-Posse (RFO) que garante que a linha existe na cache L1 e está no estado I na outra cache L1. Intel Core Duo estende esse protocolo para levar em conta o caso quando há múltiplos chips Core Duo organizados como um sistema multiprocessador simétrico (SMP). O controlador da cache L2 permite que o sistema diferencie entre uma situação onde os dados são compartilhados entre dois núcleos locais, mas não com o restante do mundo, e uma situação onde os dados são compartilhados por uma ou mais caches na pastilha assim como por um agente no barramento externo (pode ser outro processador). Quando um núcleo emite uma RFO, se a linha é compartilhada apenas pela outra cache dentro da pastilha, podemos resolver a RFO internamente muito rápido, sem ir para o barramento externo. Apenas se a linha é compartilhada com outro agente no barramento externo temos que emitir a RFO externamente.

A **interface de barramento** conecta-se com o barramento externo, conhecido como barramento frontal, o qual se conecta com a memória principal, controladores de E/S e outros chips processadores.



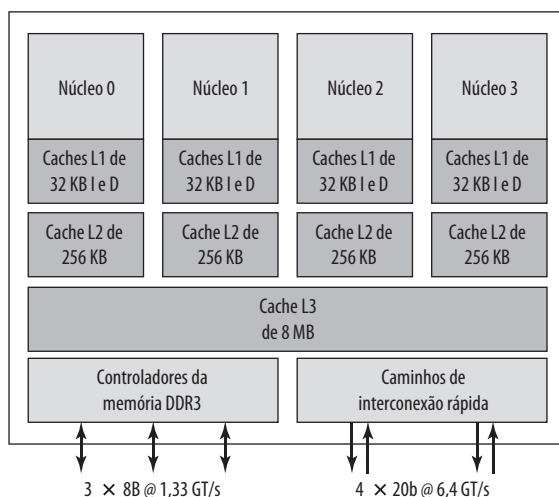
Intel Core i7

O Intel Core i7, introduzido em novembro de 2008, implementa quatro processadores x86 SMT, cada um com uma cache L2 dedicada e com uma cache L3 compartilhada (Figura 18.8d).

A estrutura geral do Intel Core i7 é mostrada na Figura 18.10. Cada núcleo tem a sua **cache L2 dedicada** e quatro núcleos compartilham a **cache L3** de 8 MB. Um mecanismo que a Intel usa para tornar suas caches mais eficientes é pré-busca, onde o hardware analisa padrões de acesso à memória e tenta preencher as caches de forma especulativa com dados que provavelmente serão requisitados logo. É interessante comparar o desempenho desta organização da cache no chip de três níveis com uma organização comparável de dois níveis da Intel. A Tabela 18.1 mostra a latência de acesso à cache em termos de ciclos de clock para dois sistemas multicore da Intel executando na mesma frequência de clock. Core 2 Quad tem uma cache L2 compartilhada, semelhante ao Core Duo. Core i7 melhora o desempenho da cache L2 com uso de caches L2 dedicadas e provê um acesso relativamente rápido à cache L3.

O chip Core i7 suporta duas formas de comunicação externa com outros chips. O **controlador de memória DDR3** traz o controlador de memória para a memória² principal DDR para o chip. A interface suporta três canais com tamanho de 8 bytes para um barramento total de 192 bits, para uma taxa de dados agregada até 32 GB/s. Com o controlador de memória no chip, o barramento frontal é eliminado.

Figura 18.10 Diagrama de blocos do Intel Core i7



2 Memória RAM síncrona DDR é discutida no Capítulo 5.

Tabela 18.1 Latência de cache (em ciclos de clock)

CPU	Frequência de clock	Cache L1	Cache L2	Cache L3
Core 2 Quad	2,66 GHz	3 ciclos	15 ciclos	—
Core i7	2,66 GHz	4 ciclos	11 ciclos	39 ciclos

O **caminho de interconexão rápida** (QPI, do inglês *quick-path interconnect*) é uma especificação de interconexão elétrica ponto a ponto, com coerência de cache para processadores e chipsets da Intel. Ele possibilita comunicação de alta velocidade entre chips de processadores conectados. A ligação QPI opera a 6,4 GT/s (transferências por segundo). A 16 bits por transferência, isso atinge até 12,8 GB/s e, como ligações QPI envolvem pares bidirecionais dedicados, a largura de banda total é 25,6 GB/s.



18.5 ARM11 MPCore

O ARM11 MPCore é um produto multicore baseado na família de processadores ARM11. O ARM11 MPCore pode ser configurado com até quatro processadores, onde cada um tem as suas próprias caches L1 de instruções e dados, por chip. A Tabela 18.2 lista as opções de configuração para o sistema, incluindo os valores padrão.

A Figura 18.11 apresenta um diagrama de bloco de ARM11 MPCore. Os principais elementos do sistema são:

- **Controlador distribuído de interrupções** (DIC, do inglês *distributed interrupt controller*): lida com detecção de interrupções e priorização de interrupções. O DIC distribui interrupções para processadores individuais.
- **Temporizador**: cada CPU tem o seu próprio temporizador privado que gera interrupções.
- **Watchdog**: emite avisos de alertas no caso de falhas de software; se o watchdog estiver habilitado, ele é ajustado para um valor predeterminado e conta até 0. Ele é reiniciado periodicamente. Se o valor do watchdog chegar a zero, um alerta é emitido.
- **Interface de CPU**: lida com confirmação de interrupções, mascaramento de interrupções e reconhecimento da conclusão de interrupções.
- **CPU**: um processador ARM11 único. CPUs individuais são referidas como **CPUs MP11**.
- **Unidade vetorial de ponto flutuante** (VFP, do inglês *vector floating-point*): um coprocessador que implementa operações de ponto flutuante em hardware.
- **Cache L1**: Cada CPU tem sua própria cache L1 de dados e cache L1 de instruções.
- **Unidade de controle de monitoramento** (SCU, do inglês *snoop control unit*): responsável por manter a coerência entre caches L1 de dados.

Tabela 18.2 Opções de configuração de ARM11 MPCore

Recurso	Intervalo de opções	Valor padrão
Processadores	1 a 4	4
Tamanho da cache de instruções por processador	16 KB, 32 KB ou 64 KB	32 KB
Tamanho da cache de dados por processador	16 KB, 32 KB ou 64 KB	32 KB
Portas mestres	1 ou 2	2
Tamanho do barramento de interrupções	0 a 224 em incrementos de 32 pinos	32 pinos
Coprocessador vetorial de ponto flutuante (VFP) por processador	Incluído ou não	Incluído



Tratamento de interrupções

O DIC recebe interrupções de um grande número de origens. Ele provê:

- Mascaramento de interrupções.
- Priorização de interrupções.
- Distribuição de interrupções para CPUs MP11-alvo.
- Rastreamento do estado das interrupções.
- Geração de interrupções pelo software.

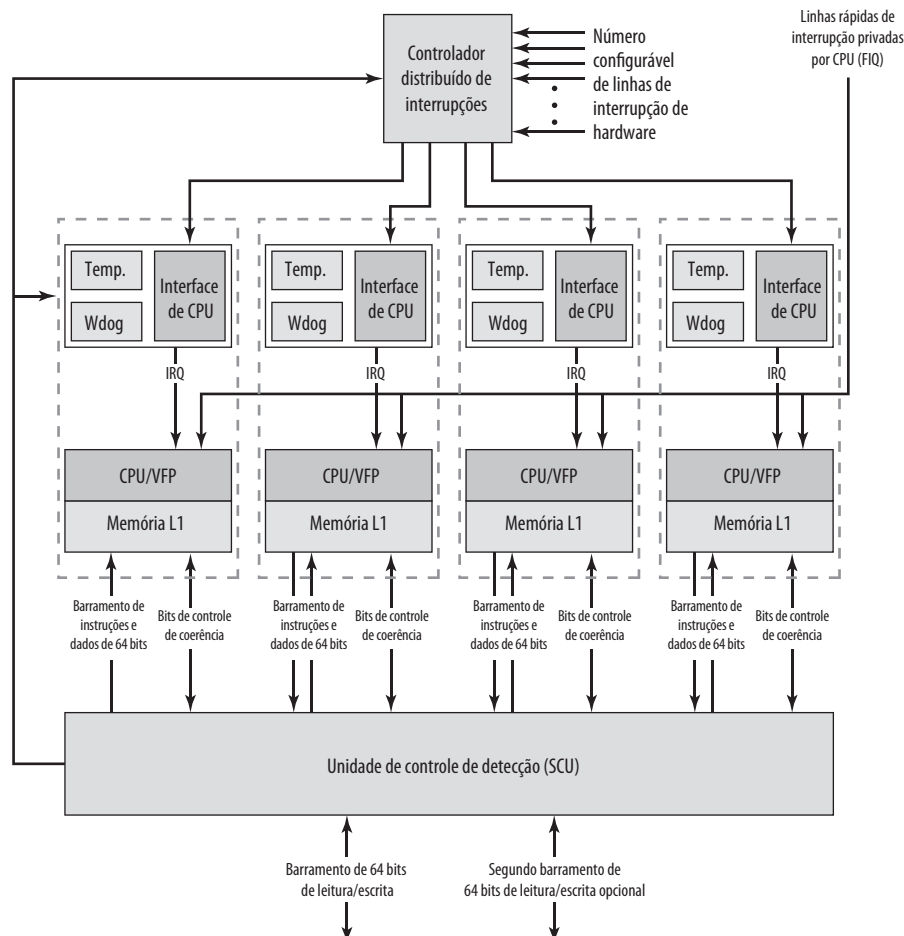
O DIC é uma unidade funcional única que é colocada no sistema ao lado das CPUs MP11. Isso possibilita que uma série de interrupções suportadas no sistema sejam independentes do projeto da CPU MP11. DIC é mapeado em memória, isto é, registradores de controle de DIC são definidos com relação a um endereço base da memória principal. O DIC é acessado pela CPUs MP11 usando uma interface privada por meio da SCU.

O DIC é projetado para satisfazer dois requisitos funcionais:

- Fornecer um meio de rotear uma requisição de interrupção para uma única CPU ou várias CPUs, conforme necessário.
- Fornecer um meio para comunicação entre processadores para que uma thread em uma CPU possa causar uma atividade em outra thread em outra CPU.

Como um exemplo que faz uso dos dois requisitos, considere uma aplicação multithread que possui threads executando em vários processadores. Admita uma aplicação que aloca alguma memória virtual. Para manter

Figura 18.11 Diagrama de blocos do processador ARM11 MPCore



consistência, o sistema operacional precisa atualizar as tabelas de tradução de memória em todos os processadores. O SO poderia atualizar as tabelas no processador onde ocorreu a alocação da memória virtual e depois emitir uma interrupção para todos os outros processadores que executam a aplicação. Outros processadores poderiam, então, usar a ID dessa interrupção para determinar que eles precisam atualizar suas tabelas de tradução de memória.

O DIC pode encaminhar uma interrupção para uma ou mais CPUs de três maneiras:

- Uma interrupção pode ser direcionada apenas para um processador específico.
- Uma interrupção pode ser direcionada para um grupo definido de processadores. O MPCore enxerga o primeiro processador para aceitar a interrupção, normalmente o menos ocupado, como sendo o mais bem posicionado para tratar a interrupção.
- Uma interrupção pode ser direcionada para todos os processadores.

Do ponto de vista do software que executa em uma determinada CPU, o SO pode gerar uma interrupção para todos menos para si mesmo ou para outras CPUs específicas. Para a comunicação entre threads que executam em CPUs diferentes, o mecanismo de interrupção é normalmente combinado com a memória compartilhada para passagem de mensagens. Desta forma, quando uma thread é interrompida por uma interrupção de comunicação entre processadores, ela lê do bloco apropriado da memória compartilhada para obter uma mensagem da thread que disparou a interrupção. Um total de 16 IDs de interrupções por CPU está disponível para a comunicação entre processadores.

Do ponto de vista de uma CPU MP11, uma interrupção pode estar:

- **Inativa:** é aquela que não está confirmada ou que foi totalmente processada em um ambiente de multiprocessamento por essa CPU, mas que ainda pode estar pendente ou ativa em algumas CPUs para as quais está destinada e por isso pode não ter sido retirada da origem da interrupção.
- **Pendente:** é aquela que foi confirmada e para qual o processamento não começou nessa CPU.
- **Ativa:** uma interrupção ativa é aquela que foi iniciada por essa CPU, porém o processamento não está completo. Uma interrupção ativa pode ser substituída quando uma interrupção nova, de prioridade maior, interrompe o processamento da CPU MP11.

As interrupções vêm das seguintes origens:

- **Interrupções entre processadores** (*interprocessor interrupts — IPI*): cada CPU possui interrupções privadas, de ID0 até ID15, que podem ser disparadas pelo software. A prioridade de uma IPI depende da CPU de destino, não da CPU da origem.
- **Temporizador privado e/ou interrupções de watchdog:** estas usam IDs de interrupção 29 e 30.
- **Linha FIQ legada:** no modo legado de IRQ, o pino legado FIQ, um por CPU base, não utiliza a lógica do distribuidor de interrupção e envia diretamente as requisições de interrupção para dentro da CPU.
- **Interrupções de hardware:** interrupções do hardware são disparadas por eventos programáveis em linhas de entrada de interrupção associadas. As CPUs podem suportar até 224 linhas de entrada de interrupção. Interrupções de hardware iniciam em ID32.

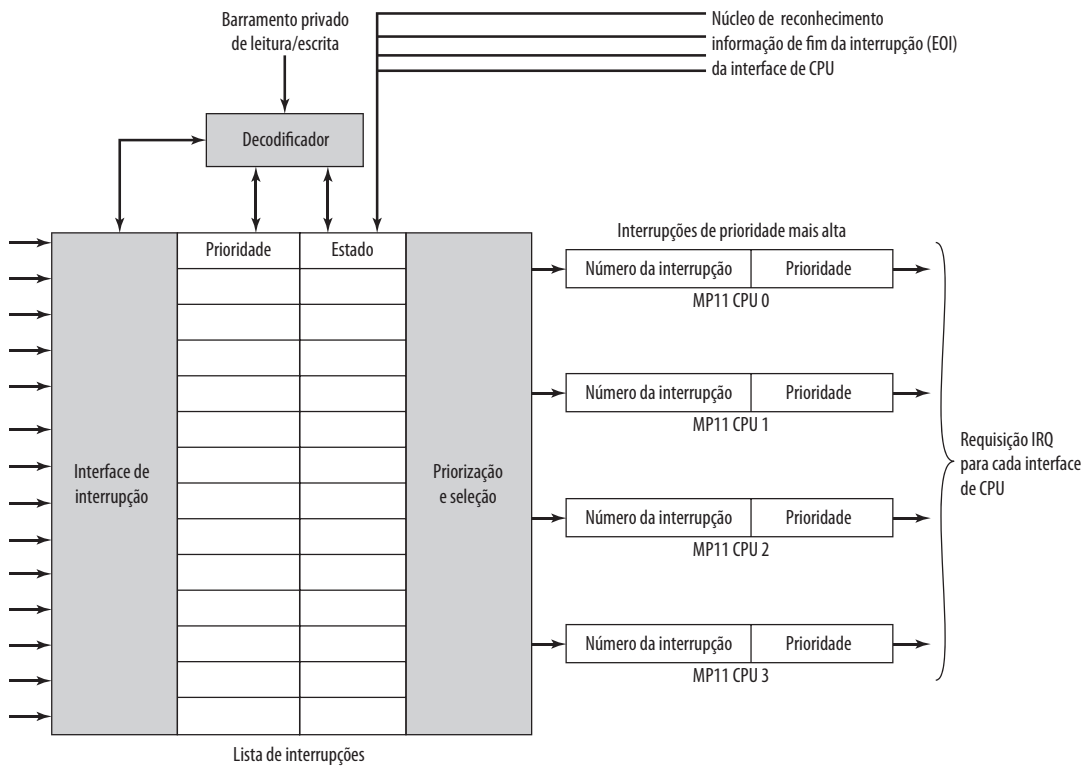
A Figura 18.12 é um diagrama de blocos de DIC. O DIC é configurável para suportar entre 0 e 255 entradas de interrupções de hardware. Ele mantém uma lista de interrupções, mostrando a sua prioridade e estado. O distribuidor de interrupções transmite para cada interface da CPU a interrupção pendente mais alta para essa interface. Ele recebe de volta a informação que a interrupção foi recebida e pode então mudar o estado da interrupção correspondente. A interface de CPU também transmite a informação de fim da interrupção (*end of interrupt information — EOI*), o qual habilita o distribuidor de interrupções para atualizar o estado dessa interrupção de ativa para inativa.



Coerência de cache

A unidade de controle de monitoramento de MPCore (*MPCore's snoop control unit* — SCU) é projetada para resolver a maioria dos gargalos tradicionais relacionados com acesso a dados compartilhados e limitações de escalabilidade introduzidas pelo tráfego de coerência.

O esquema de coerência da cache L1 é baseado no protocolo MESI descrito no Capítulo 17. A SCU monitora os dados compartilhados das operações para otimizar migração de estados do MESI. Ela introduz três tipos de otimização: intervenção direta de dados, tags duplicadas de RAMs e linhas migratórias.

Figura 18.12 Diagrama de blocos do distribuidor de interrupções

Intervenção direta de dados (*direct data intervention — DDI*) possibilita copiar dados limpos de cache L1 de dados de uma CPU para cache L1 de dados de outra CPU sem acessar a memória externa. Isto reduz a atividade de leitura após a leitura de cache L1 para cache L2. Assim, uma falha de cache L1 local é resolvida em uma cache L1 remota em vez de acesso à cache L2 compartilhada.

Lembre que a posição da memória principal de cada linha dentro de uma cache é identificada por um rótulo (tag) para essa linha. Os tags podem ser implementadas como um bloco separado de RAM do mesmo tamanho do número de linhas na cache. Na SCU, **tags duplicadas de RAM** são versões duplicadas de tags de RAM da L1 usadas pela SCU para verificar a disponibilidade de dados antes de enviar comandos de coerência para CPUs relevantes. Comandos de coerência são enviados apenas para CPUs que precisam atualizar sua cache de dados coerente. Isto reduz o consumo de energia e o impacto em desempenho por causa da detecção e manipulação de cache de cada processador em cada atualização de memória. Ter dados das marcações disponíveis localmente permite que a SCU limite a manipulação de cache para processadores que possuem linhas de cache em comum.

O recurso de **linhas migratórias** permite mover dados sujos de uma CPU para outra sem escrever na L2 e ler dados de volta a partir da memória externa. A operação pode ser descrita conforme segue. Em um protocolo MESI típico, quando um processador tem uma linha modificada e outro processador tenta ler essa linha, ocorrem as seguintes ações:

1. O conteúdo da linha é transferido da linha modificada para o processador que iniciou a leitura.
2. O conteúdo da linha é lido de volta para memória principal.
3. A linha é colocada no estado compartilhado em ambas as caches.

O MPCore SCU lida com esta situação de forma diferente. O SCU monitora o sistema por uma linha migratória. Se um processador tem uma linha modificada e outro processador lê e depois escreve nela, SCU supõe que tal posição sofrerá a mesma operação no futuro. Quando essa operação começar novamente, a SCU automaticamente

moverá a linha da cache diretamente para um estado inválido em vez de gastar energia movendo-a primeiramente para estado compartilhado. Esta otimização também faz com que o processador transfira a linha da cache diretamente para outro processador sem intervenção das operações de memória externa.



18.6 Leitura recomendada e sites Web

Dois livros que oferecem boa cobertura sobre questões deste capítulo são Olukotun, Hammond e Laudon (2007^l) e Jerraya e Wolf (2005^k). Gochman et al. (2006^j) e Mendelson (2006^m) descrevem Intel Core Duo. Fog (2008ⁿ) fornece uma descrição detalhada da arquitetura de pipeline de Core Duo.

ARM (2008^o) fornece cobertura completa sobre o pipeline de ARM Cortex-A8. Hirata e Goodacre (2007^p) e Goodacre e Sloss (2005^q) são bons artigos para uma visão geral.



Sites Web recomendados

Multicore Association: organização de fabricantes que promove o desenvolvimento e o uso de tecnologia multicore.

Principais termos, perguntas de revisão e problemas

Principais termos

Lei de Amdahl	Multicore	Superescalar
Chip multiprocessador	Multithreading simultâneo (SMT)	

Perguntas de revisão

- 18.1 Resuma a diferença entre pipeline de instruções simples, superescalar e multithreading simultâneo.
- 18.2 Dê várias razões para a escolha dos projetistas para migrar para uma organização multicore em vez de aumentar o paralelismo dentro de um único processador.
- 18.3 Por que há uma tendência para se aumentar a fração da área do chip para memória cache?
- 18.4 Relacione alguns exemplos de aplicações que se beneficiam diretamente da habilidade de aumentar rendimento com número de núcleos.
- 18.5 No nível mais alto, quais são as principais variáveis do projeto em uma organização multicore?
- 18.6 Relacione algumas vantagens de cache L2 compartilhada entre núcleos comparada com caches L2 separadas dedicadas para cada núcleo.

Problemas

- 18.1 Considere o seguinte problema. Um projetista tem um chip disponível e decide qual fração dele será dedicada para memória cache (L1, L2, L3). O restante do chip pode ser dedicado para um complexo superescalar único e/ou núcleo SMT ou vários núcleos mais simples. Defina os seguintes parâmetros:
 n = número máximo de núcleos que podem ser contidos no chip.
 k = número atual de núcleos implementados ($1 \leq k \leq n$, onde $r = n/k$ é um inteiro).
 $perf(r)$ = desempenho sequencial obtido com uso de recursos equivalentes para r núcleos para formar um processador único, onde $perf(1) = 1$.
 f = fração do software que é paralelizável por vários núcleos.

Assim, se construirmos um chip com n núcleos, esperamos que cada núcleo forneça desempenho sequencial de 1 e que, para n núcleos, seja capaz de explorar o paralelismo até um nível de n threads paralelas. De forma semelhante, se o chip tiver k núcleos, então cada núcleo deveria demonstrar um desempenho de $perf(r)$ e o chip é capaz de explorar paralelismo até um nível de k threads paralelas.

$$\text{Aumento de velocidade} = \frac{1}{\frac{1-f}{\text{perf}(r)} + \frac{f \times r}{\text{perf}(r) \times n}}$$

- a. Justifique esta modificação da lei de Amdahl.
 - b. Usando a regra de Pollack, definimos $\text{perf}(r) = \sqrt{r}$. Seja $n=16$. Queremos desenhar o aumento de velocidade como uma função de r para $f=0,5$; $f=0,9$; $f=0,975$; $f=0,99$; $f=0,999$. Os resultados estão disponíveis em um documento no site deste livro (multicore-desempenho.pdf). Que conclusões você pode tirar?
 - c. Repita a parte (b) para $n=256$.
- 18.2** O manual de referência técnica para ARM11 MPCore diz que o controlador de instruções distribuído é mapeado em memória. Isto é, os núcleos processadores usam E/S mapeada em memória para se comunicar com DIC. Lembre do Capítulo 7 que, com E/S mapeada em memória, há um espaço de endereço único para posições de memória e dispositivos de E/S. O processador trata o estado e os registradores de dados dos módulos de E/S como posições de memória e usa as mesmas instruções de máquina para acessar memória e dispositivos E/S. Com base nesta informação, qual o caminho através do diagrama de blocos da Figura 18.11 é usado para o processador de núcleo se comunicar com DIC?

Referências

- a OLUKOTUN, K. e HAMMOND, L. "The future of microprocessors". *ACM Queue*, set. 2005.
- b BORKAR, S. "Getting gigascale chips: challenges and opportunities in continuing Moore's law". *ACM Queue*, out. 2003.
- c BORKAR, S. "Thousand core chips—a technology perspective". *Proceedings, ACM/IEEE Design Automation Conference*, 2007.
- d POLLACK, F. "New microarchitecture challenges in the coming generations of cmos process technologies (keynote address)". *Proceedings of the 32nd annual ACM/IEEE International Symposium on Microarchitecture*, 1999.
- e MCDUGALL, R. "Extreme software scaling". *ACM Queue*, set. 2005.
- f MCDUGALL, R. e LAUDON, J. "Multi-core microprocessors are Here"; *login*, out. 2006.
- g REIMER, J. "Valve goes multicore". *ars technica*, nov. 2006. Disponível em: <arstechnica.com/articles/paedia/cpu/valve-multicore.ars>.
- h HARRIS, W. "Multi-core in the source engine". bit-tech.net technical paper, nov. 2006. Disponível em: <bit-tech.net/gaming/2006/11/02/Multi_core_in_the_Source_Engin/1>.
- i LEONARD, T. "Dragged kicking and screaming: source multicore". *Proceedings, Game Developers Conference 2007*, mar. 2007.
- j OLUKOTUN, K.; HAMMOND, L. e LAUDON, J. *Chip multiprocessor architecture: techniques to improve throughput and latency*. San Rafael, CA: Morgan & Claypool, 2007.
- k JERRAYA, A. e WOLF, W., eds. *Multiprocessor systems-on-chips*. San Francisco: Morgan Kaufmann, 2005.
- l GOCHMAN, S., et al. "Introduction to Intel Core Duo processor architecture". *Intel Technology Journal*, mai. 2006.
- m MENDELSON, A., et al. "CMP implementation in systems based on the Intel Core Duo processor". *Intel Technology Journal*, mai. 2006.
- n FOG, A. *The microarchitecture of Intel and AMD CPUs*. Copenhagen University College of Engineering, 2008. Disponível em: <www.agner.org/optimize>.
- o ARM LIMITED. *ARM11 MPCore processor technical reference manual*. ARM DDI 0360E, 2008. Disponível em: <www.arm.com>.
- p HIRATA, K. e GOODACRE, J. "ARM MPCore: the streamlined and scalable ARM11 processor core". *Proceedings, 2007 Conference on Asia South Pacific Design Automation*, 2007.
- q GOODACRE, J. e SLOSS, A. "Parallelism and the ARM instruction set architecture". *Computer*, jul. 2005.